



文件读写

12.1 学生成绩文件统计

12.2 用户信息加密和校验

12.3 文件综合应用：资金账户管理

本章要点

- 什么是文件？C文件是如何存储的？
- 什么是文件缓冲系统？工作原理如何？
- 什么是文本文件和二进制文件？
- 怎样打开、关闭文件？
- 怎样编写文件读写程序？
- 怎样编写程序，实现简单的数据处理？

12.1 学生成绩文件统计

【例12-1】有5位学生的计算机等级考试成绩被事先保存在数据文件C:\f12-1.txt(C盘根目录下的文件f12-1.txt，需事先准备好该文件)中，包括学号、姓名和分数，文件内容如下：

301101 Zhangwen 91

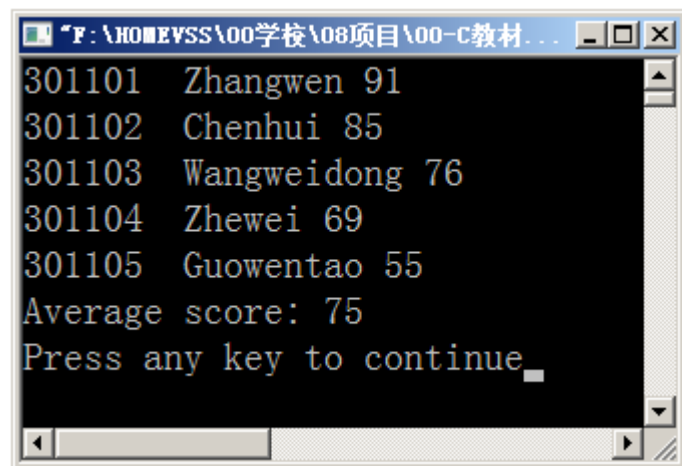
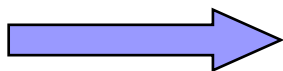
301102 Chenhui 85

301103 Wangweidong 76

301104 Zhengwei 69

301105 Guowentao 55

请读出文件的所有内容显示到屏幕，并输出平均分。



例12-1 源程序

```
#include <stdio.h>
#include <stdlib.h>
#include <process.h>
int main(void)
{
    FILE *fp;                                /*1.定义文件指针*/
    long num;  char stname[20];
    int i, score; int avg_score = 0;
    if((fp=fopen("c:\\f12-1.txt","r")) == NULL) /*2.打开文件*/
    {
        printf("File open error!\n");
        exit(0);
    }
}
```

续下页....

例12-1 源程序

接上页.....

/*3.文件处理（逐个读入和处理数据）*/

```
for(i=0;i<5;i++)
```

```
{
```

```
    /*从文件读入成绩保存到变量*/
```

```
    fscanf(fp,"%ld%s%d",&num,stype,&score);
```

```
    avg_score += score; /*统计总分*/
```

```
    /*输出成绩到屏幕*/
```

```
    printf("%ld %s %d\n",num,stype,score);
```

```
}
```

```
/*输出平均分到屏幕*/
```

```
printf("Average score: %d\n", avg_score/5);
```

```
if(fclose(fp)){
```

/*4.关闭文件*/

```
    printf( "Can not close the file!\n" );
```

```
    exit(0);
```

```
}
```

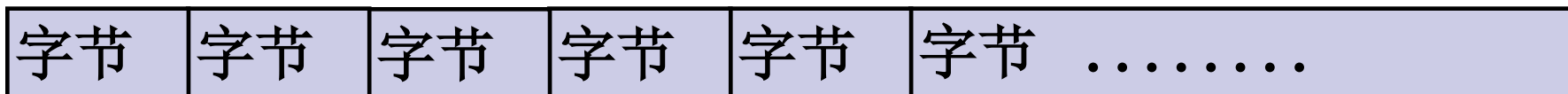
```
return 0;
```

```
}
```

12.1.2 文件的概念

- 文件：操作系统中的文件是指驻留在外部介质（如磁盘等）中的一个有序数据集。
- 各种类型的文件
 - 程序文件：源文件、目标程序、可执行程序
 - 数据文件（输入/输出）：文本文件、图像文件、声音文件、可执行文件等
- 文件的特点：
 - 数据永久保存；数据长度不定；数据按顺序存取

12.1.3 文本文件和二进制文件



C 语言中的文件是数据流(由一个个的字节数据组成)

文件的两种数据形式:

- **ASCII码** (文本文件 **text stream**) **字符流**
- **二进制码** (二进制文件 **binary stream**) **二进制流**

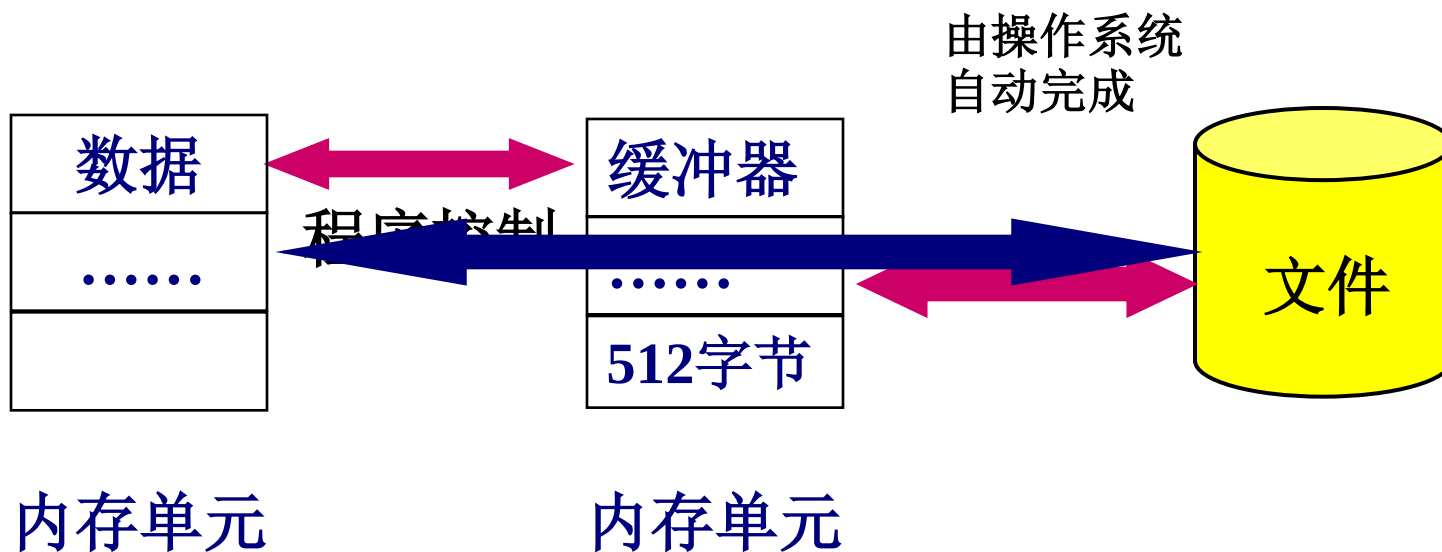
二进制文件是直接吧内存数据以二进制形式保存。

例如, 整数**1234**

- 文本文件保存: **49 50 51 52** (4个字符)
- 二进制文件保存: **04D2** (**1234**的二进制数)

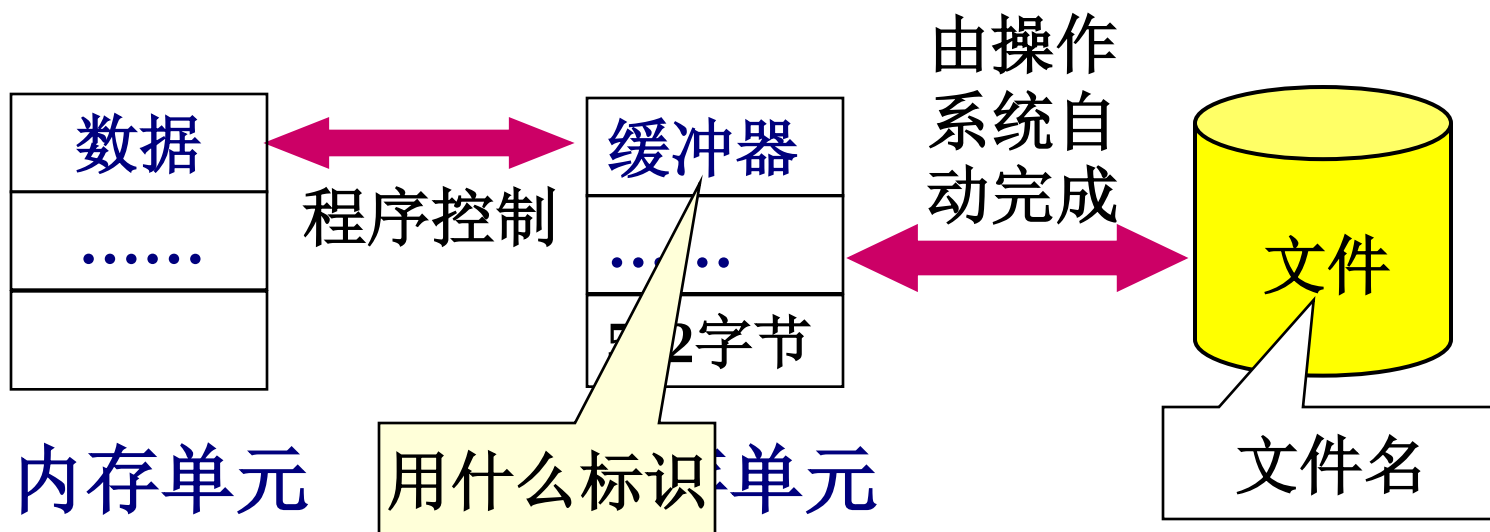
12.1.4 缓冲文件系统

由于磁盘速度慢
直接把数据写到磁盘效率很低



12.1.4 缓冲文件系统

- 向磁盘输出数据：数据 $\longrightarrow$ 缓冲区，装满缓冲区后 $\longrightarrow$ 磁盘文件。
- 从磁盘读入数据：先一次性从磁盘文件将一批数据输入到缓冲区，然后再从缓冲区逐个读入数据到变量。



缓冲文件与文件类型指针

用文件指针指示文件缓冲区中具体读写的位置

FILE *fp;



使用多个文件时，每个文件都有缓冲区，用不同同时的文件指针分别指示。

12.1.5 文件结构与文件类型指针

1. 文件结构与自定义类型typedef

FILE: 结构类型， 用 **typedef** 定义(见stdio.h)

```
typedef struct{
    short          level;          /* 缓冲区使用量 */
    unsigned       flags;          /* 文件状态标志 */
    char           fd;             /* 文件描述符 */
    short          bsize;          /* 缓冲区大小 */
    unsigned char  *buffer;        /* 文件缓冲区的首地址 */
    unsigned char  *curp;          /* 指向文件缓冲区的工作指针 */
    unsigned char  hold;           /* 其他信息 */
    unsigned       istemp;
    short          token;
} FILE;
```

自定义类型（**typedef**）：

- 将C语言中的已有类型（包括已定义过的自定义类型）重新命名
- 新的名称可以代替已有数据类型
- 常用于简化对复杂数据类型定义的描述

typedef <已有类型名> <新类型名>;

typedef int **INTEGER**;

int i, j; <====> **INTEGER** i, j;

typedef int* **POINT**;

int* p1; <====> **POINT** p1;

自定义类型（typedef）的使用方法

- 定义变量
- 变量名 → 新类型名
- 加上 **typedef**
- 用新类型名定义变量

```
int i  
int → INTEGER  
typedef  int INTEGER  
INTEGER i;
```

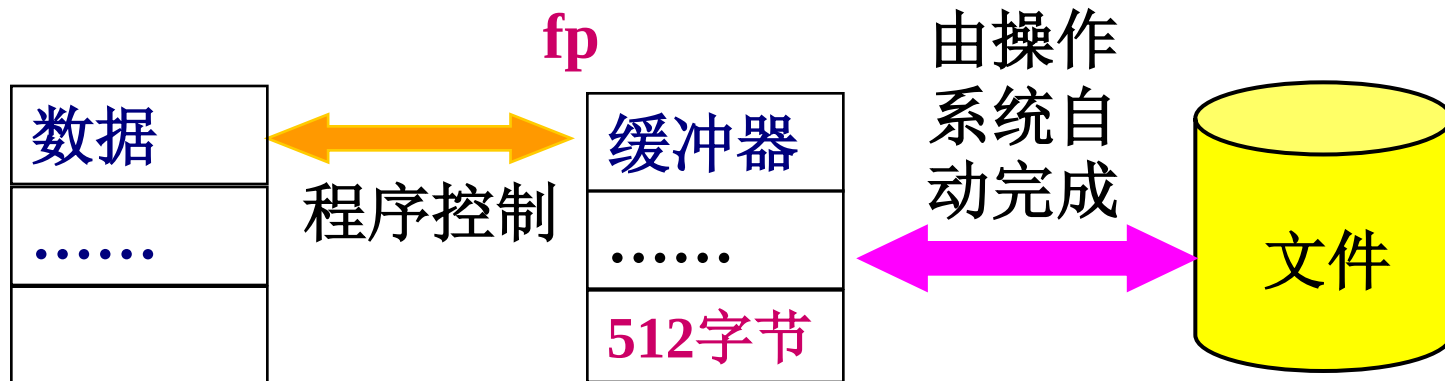
```
int num[10]  
int NUM[10]  
typedef  int NUM[10]  
NUM  a <===> int a[10]
```

2. 文件类型指针

FILE * fp

如何使**fp**与具体文件挂钩？

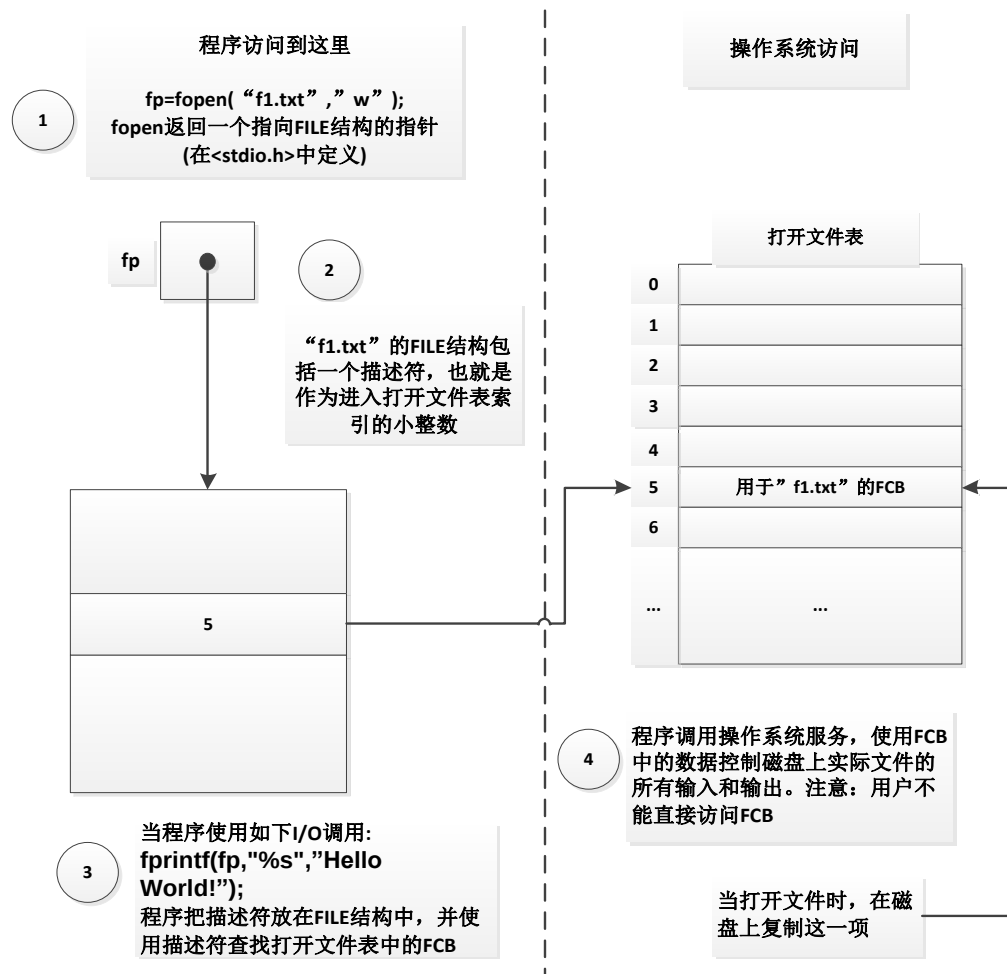
指向文件缓冲区，通过移动指针实现对文件的操作



同时使用多个文件时，每个文件都有缓冲区，用不同的文件指针分别指示。

12.1.6 文件控制块FCB

- 文件控制块FCB（File Control Block）
- OS中对文件的操作控制通过FCB，处理的是FCB列表
- 一个文件对应一个FCB
- 文件缓冲区由程序中fopen语句动态创建
- 打开文件时，FCB的内容信息被复制到文件缓冲区保存
- 用文件指针指向文件缓冲区实现对文件数据的访问



12.1.7 文件处理步骤

■ 四个步骤:

- ① 定义文件指针
- ② 打开文件: 文件指针指向磁盘文件缓冲区
- ③ 文件处理: 文件读写操作
- ④ 关闭文件

12.2 用户信息加密和校验

【例12-2】 为了保障系统安全，通常采取用户帐号和密码登录系统。系统用户信息存放在一个文件中，系统帐号名和密码由若干字母与数字字符构成，因安全需要文件中的密码不能是明文，必须要经过加密处理。请编程实现：输入5个用户信息（包含帐号名和密码）并写入文件f12-2.dat。要求文件中每个用户信息占一行，帐号名和加密过的密码之间用一个空格分隔。密码加密算法：对每个字符ASCII码的低四位求反，高四位保持不变（即将其与15进行异或）。

12.2.1 程序解析

```
#include <stdio.h>
#include <string.h>
struct sysuser{/*用户帐号信息结构*/
    char username[20];
    char password[8];
};
```

```
int main(void)
```

```
{
    FILE *fp;    /*1.定义文件指针*/
    int i;
    void encrypt(char *pwd);
    struct sysuser su;
    /*2.打开文件，进行写入操作*/
    if((fp=fopen("f12-2.txt","w")) == NULL){
        printf("File open error!\n"); exit(0);
    }
    for(i=1;i<=5;i++){/*3. 将5位用户帐号信息写入文件*/
        printf("Enter %i th sysuser(name password):",i);
        scanf("%s%s",su.username,su.password); /*输入用户名和密码 */
        encrypt(su.password);    /*进行加密处理*/
        fprintf(fp,"%s %s\n",su.username,su.password); /*写入文件*/
    }
    if(fclose(fp)){ /*4.关闭文件*/
        printf("Can not close the file!\n"); exit(0);}
    return 0;
}
```

```
/*加密算法*/
```

```
void encrypt(char *pwd)
```

```
{
    int i;
    /*与15（二进制码是00001111）异或，实现低四位取反，高四位保持不变*/
    for(i=0;i<strlen(pwd);i++)
        pwd[i] = pwd[i] ^ 15;
}
```

12.2.2 打开文件和关闭文件

```
if((fp=fopen("f12-2.txt","w")) == NULL){  
    printf("File open error!\n"); exit(0);  
}
```

fopen("文件名", "文件打开方式")

- 使文件指针与相应文件实体对应起来
- 程序对文件指针进行操作，即**fp**代表磁盘文件

■ 函数**fopen()** 的返回值

- 执行成功，则返回包含文件缓冲区等信息的**FILE**型地址，赋给文件指针**fp**
- 不成功，则返回一个**NULL**（空值）

exit(0): 关闭所有打开的文件，并终止程序的执行

参数**0**表示程序正常结束；非**0**参数通常表示不正常的程序结束

文件打开方式

```
fp=fopen("f12-2.txt","w")
```

■ 文件打开方式参数表

文 本 文 件 (ASCII)		二 进 制 文 件(Binary)	
使用方式	含 义	使用方式	含 义
“ r ”	打开只读文件	“ rb ”	打开只读文件
“ w ”	建立只写新文件	“ wb ”	建立只写新文件
“ a ”	打开添加写文件	“ ab ”	打开添加写文件
“ r+ ”	打开读/写文件	“ rb+ ”	打开读/写文件
“ w +”	建立读/写新文件	“ wb+ ”	建立读/写新文件
“ a +”	打开读/写文件	“ ab+ ”	打开读/写文件

文件读写与打开方式

if 读文件

指定的文件必须存在，否则出错；

if 写文件(指定的文件可以存在，也可以不存在)

if 以 "w" 方式写

if 该文件已经存在

原文件将被删去重新建立；

else

按指定的名字新建一个文件；

else if 以 "a" 方式写

if 该文件已经存在

写入的数据将被添加到指定文件原有数据的后面，不会删去原来的内容；

else

按指定的名字新建一个文件（与“w”相同）；

if 文件同时读和写

使用 "r+"、"w+" 或 "a+" 打开文件

关闭文件

```
if( fclose(fp) ){  
    printf( "Can not close the file!\n" );  
    exit(0);  
}
```

fclose(文件指针)

- 把缓冲区中的数据写入磁盘扇区，确保写文件的正常完成
- 释放文件缓冲区单元和**FILE**结构体，使文件指针与具体文件脱钩。

函数**fclose()** 的返回值

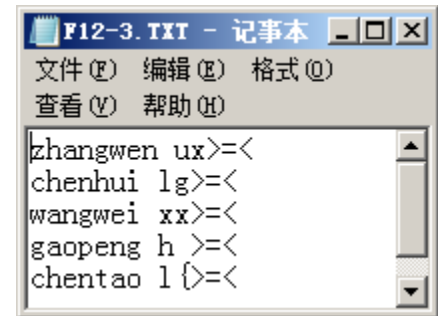
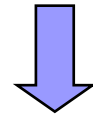
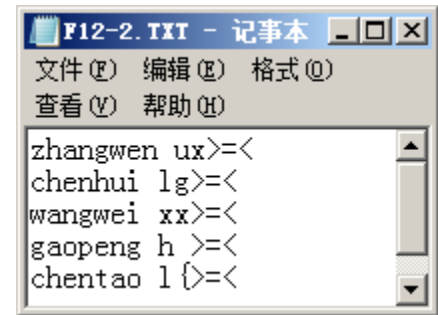
- 返回**0**：正常关闭文件
- 返回非**0**：无法正常关闭文件

12.2.3 文件读写

【例12-3】 复制用户文件。将例12-2的用户信息文件f12-2.txt文件备份一份，取名为文件f12-3.txt。说明：运行程序前请将文件f12-2.txt与源程序放在同一目录下。

例12-3 源程序

```
#include <stdio.h>
int main(void)
{   FILE *fp1,*fp2;
    char ch;
    if(( fp1 = fopen( "f12-2.txt", "r" )) == NULL){
        printf(" File open error!\n" );
        exit(0);
    }
    if(( fp2 = fopen( "f12-3.txt", "w" )) == NULL){
        printf(" File open error!\n" );
        exit(0);
    }
    while( !feof( fp1 ) ){
        ch = fgetc( fp1 );
        if(ch!=EOF) fputc(ch, fp2);
    }
    /*关闭文件f12-2.txt */
    if(fclose(fp1)){ printf("Can not close the file!\n"); exit(0);}
    /*关闭文件f12-3.txt */
    if(fclose(fp2)){ printf("Can not close the file!\n"); exit(0);}
    return 0;
}
```



打开多个文件

```
if((fp1 = fopen("f12-2.txt", "r")) == NULL){  
    printf("File open error!\n");  
    exit(0);  
}  
if((fp2=fopen("f12-3.txt", "w")) == NULL){  
    printf("File open error!\n");  
    exit(0);  
}
```

C语言允许同时打开多个文件

- 不同的文件对应不同的文件指针
- 不允许同一个文件在关闭前再次打开

文件读写函数

- 字符读写函数: `fgetc()` / `fputc()`
- 字符串读写函数: `fputs()` / `fgets()`
- 格式化读写函数: `fscanf()` / `fprintf()`
- 二进制读写函数: `fread()` / `fwrite()`
- 其他相关函数:
 - 检测文件结尾函数 `feof()`
 - 检测文件读写出错函数 `ferror()`
 - 清除末尾标志和出错标志函数 `clearerr()`
 - 文件定位的函数 `fseek()`、`rewind()`、`ftell()`

1. 字符读写函数fgetc和fputc

```
while( !feof(fp1) ){  
    ch = fgetc( fp1 );  
    if(ch!=EOF) fputc(c, fp2);  
}
```

■ 函数fgetc()

ch = fgetc(fp);

从fp所指示的磁盘文件上读
入一个字符到ch

□ 区分键盘字符输入函数
getchar()

■ 函数fputc()

□ **fputc(ch, fp);**

把一个字符 **ch** 写到 **fp** 所指
示的磁盘文件上

□ 返回值

■ **-1 (EOF):** 写文件失败

■ **ch:** 写文件成功

2. 字符串方式读写函数fgets和fputs

- 函数fputs()

fputs(s, fp);

用来向指定的文本文件写入一个字符串

- **s**: 要写入的字符串，结束符' \0'不写入文件。

- 函数返回值

- 执行成功，函数返回所写的最后一个字符
- 否则，函数返回**EOF**

字符串方式读写函数fgets和fputs

■ 函数fgets()

fgets(s, n, fp);

从文本文件中读取字符串

- **s**: 可以是字符数组名或字符指针；**n**: 指定读入的字符个数；**fp**: 文件指针
- 函数被调用时，最多读取**n-1**个字符，并将读入的字符串存入**s**所指向内存地址开始的**n-1**个连续的内存单元中。

当函数读取的字符达到指定的个数，或接收到换行符，或接收到文件结束标志**EOF**时，将在读取的字符后面自动添加一个'**\0**'字符；若有换行符，则将换行符保留（换行符在'**\0**'字符之前）；若有**EOF**，则不保留

□ 函数返回值

- 执行成功，返回读取的字符串；
- 如果失败，则返回空指针，这时，**s**的内容不确定

例12-4

例12-2的f12-2.txt文件保存着系统用户信息，编写一个函数checkUserValid()用于登录系统时校验用户的合法性。检查方法是：

- 在程序运行时输入用户名和密码，然后在用户文件中查找该用户信息，如果用户名和密码在文件中找到，则表示用户合法，返回1，否则返回0。
- 程序运行时，输入一个用户名和密码，调用checkUserValid()函数，如果返回1，则提示“Valid user!”，否则输出“Invalid user!”。

提示：合法性检查的规则。由于文件中的用户名和密码按行存取，把一行看作整体得字符串s1，将输入的用户名和密码加密后生成另一个字符串s2，然后通过比较s1和s2，来确定文件中是否存在用户。

例12-4源程序

```
/*校验用户信息的合法性，成功返回1，否则返回0*/
int checkUserValid(struct sysuser *psu)
{
    FILE *fp;
    char usr[30],usr1[30],pwd[10];
    int check=0;      /*检查结果变量，初始化为0*/
    /*连接生成待校验字符串*/
    strcpy(usr,psu->username); /*复制psu->username到usr1 */
    strcpy(pwd,psu->password); /*复制psu->password到pwd */
    encrypt(pwd);      /*调用例12-2的encrypt对密码进行加密*/
    /*连接usr、空格、pwd和\n构成新字符串usr，用于在文件中逐行检查*/
    strcat(usr, " "); strcat(usr,pwd); strcat(usr,"\n");
    /*打开文件"f12-2.txt"读入*/
    if((fp=fopen("f12-2.txt","r"))==NULL){
        printf("File open error!\n");  exit(0);  }
    /*从文件读入用户信息数据，遍历判断是否存在*/
    while(!feof(fp)){
        fgets(usr1,30,fp);      /*读入一行用户信息作为一个字符串到usr1*/
        if(strcmp(usr,usr1)==0){ /*比较判断usr与usr1是否相同*/
            check=1; break;  }
        }
    if(fclose(fp)){ printf("Can not close the file!\n"); exit(0); } /*关闭文件*/
    return check;
}
```

3. 格式化文件读写fscanf和fprintf

- **fscanf** (文件指针, 格式字符串, 输入表);
 - **fprintf** (文件指针, 格式字符串, 输出表);
- 指定格式的输入输出函数

```
FILE *fp; int n; float x;
```

```
fp = fopen("a.txt", "r");
```

```
fscanf(fp, "%d%f", &n, &x);
```

表示从文件a.txt分别读入整型数到变量n、浮点数到变量x

```
fp = fopen("b.txt", "w");
```

```
fprintf(fp, "%d%f", n, x);
```

表示把变量n和x的数值写入文件b.txt

4. 数据块读写fread()和fwrite()

- **fread(buffer, size, count, fp);**

从二进制文件中读入一个数据块到变量

- **fwrite(buffer, size, count, fp);**

向二进制文件中写入一个数据块

- ☐ **buffer:** 指针，表示存放数据的首地址；

- ☐ **size:** 数据块的字节数

- ☐ **count:** 要读写的数据块块数

- ☐ **fp:** 文件指针

12.2.4 其他相关函数

■ 函数feof()

feof(fp) ;

判断fp指针是否已经到文件末尾

函数返回值

- 1: 到文件结束位置
- 0: 文件未结束

■ 函数rewind()

rewind(FILE *fp);

定位文件指针，使文件指针指向读写文件的首地址，即打开文件时文件指针所指向的位置。

其他相关函数

- 函数 **fseek()**——用来控制指针移动

fseek(fp, offset, from);

- **offset**: 移动偏移量, long型
- **from**: 起始位置, 文件首部、当前位置和文件尾部分别对应0,1,2, 或常量 **SEEK_SET**、**SEEK_CUR**、**SEEK_END**。

例如:

fseek(fp, 20L, 0): 将文件位置指针移动到离文件首20字节处

fseek(fp, -20L, SEEK_END): 将文件位置指针移动到离文件尾部前20字节处

- 函数 **ftell()**

ftell(文件指针);

获取当前文件指针的位置, 即相对于文件开头的位移量 (字节数)

- 函数出错时, 返回-1L

其他相关函数

- **ferror()**函数：函数用来检查文件在用各种输入输出函数进行读写是否出错，若返回值为0，表示未出错，否则表示有错

调用形式为：**ferror(文件指针);**

- 文件指针必须是已经定义过的

- 函数**clearerr()**

clearerr(文件指针);

用来清除出错标志和文件结束标志，使它们为0

12.3 文件综合应用：个人资金账户管理

■ 12.3.1 顺序文件和随机文件

按照C程序对文件访问的特点来分，文件可分为顺序访问文件和随机访问文件，简称为顺序文件和随机文件。前面介绍的所有例子都进行的是顺序访问，通过使用fprintf()或fputs()函数创建的数据记录长度并不是完全一致的，这种记录长度不确定的文件访问称为顺序访问。而随机访问文件要求文件中单个记录的长度固定，可直接访问，这样速度快，并且无需通过其他记录查找特定记录。因此随机文件适合银行系统、航空售票系统、销售点系统和其他需要快速访问特定数据的事务处理系统。

12.3.2 个人资金帐户的管理

■ 要求

- 个人资金账户的信息统一放在随机文件中,该随机文件包括的数据项有记录ID、发生日期、发生事件、发生金额（正+的表示收入，负-表示支出）和余额。每记录一次收支，文件要增加一条记录，并计算一次余额。
- 程序实现3个功能，包括：**1**) 可以创建该文件并添加新收入或支出信息；**2**) 可以显示所有记录列表，得知资金账户的收支流水帐；**3**) 查询最后一条记录，获知账户最后的余额。

cashbox.dat文件的部分内容

LogID	CreateDate	Note	Charge	Balance
1	2006-06-01	alimony	500.00	500.00
2	2006-06-08	shopping	-300.00	200.00
3	2006-06-15	shopping	-60.00	140.00
4	2006-06-20	workingpay	200.00	340.00
5	2006-08-01	scholarship	1000.00	1340.00

.....

本章总结

- 文件的概念
 - 文本文件和二进制文件
 - 文件缓冲系统
 - 文件结构，文件指针，自定义类型
- 文件的打开与关闭
 - 文件处理实现过程
- 文件读写操作与常用文件操作函数
- 文件综合应用

- 1) 掌握文件的概念，文件缓冲系统的基本原理，文件读写操作实现的基本过程；
- 2) 能熟练使用文件进行编程。
- 3) 掌握常用的文本文件读写操作函数。
- 4) 了解顺序文件和随机文件的应用。